

Newsroom Pulse: A Near-Real-Time Big-Data Architecture for Multilingual News Sentiment Monitoring

Newsroom Pulse Project Team

International and Nepali News Monitoring

Abstract. This report describes the design of Newsroom Pulse, a near-real-time pipeline for source-aware monitoring of international and Nepali news headlines. The system objective is to collect RSS records, preserve publisher and regional provenance, suppress exact-title duplicates before external inference, estimate headline-level sentiment, persist enriched records, and present recent stories and aggregates. The configured source inventory contains 19 feeds: 12 international and 7 Nepali. A Python producer publishes normalized JSON records to Apache Kafka; Apache Spark Structured Streaming consumes bounded micro-batches, performs persistent and within-batch title checks, and submits region-specific batches of at most five headline titles to Cloudflare Workers AI, with Gemini as a fallback. MongoDB stores article metadata and analysis fields, while a Streamlit application queries the collection for regional views and time-window summaries. The report defines the problem, data contract, storage strategy, component rationale, processing sequence, and evaluation protocol. It distinguishes configured design parameters from empirical performance: the implementation has a single Spark worker, driver-side micro-batch collection, process-local provider pacing, and no configured broker replication beyond a factor of one. Consequently, throughput, latency, and sentiment-quality outcomes require separate measurement and are not inferred from configuration alone.

1. Introduction and Problem Definition

1.1 Problem Context

Digital newsrooms and media-monitoring teams receive reports from a heterogeneous set of publishers. A source-by-source manual workflow makes it difficult to maintain timely situational awareness, preserve source provenance, identify repeated headlines, and compare coverage across geographic or linguistic groups. The problem is particularly relevant when Nepali headlines use Devanagari, Romanized Nepali, English, or mixtures of these forms.

1.2 Objectives and Scope

Newsroom Pulse addresses this problem as a data-engineering and decision-support task. Its objective is to convert publisher-supplied RSS items into source-attributed records, use a stream-processing path to remove exact-title duplicates before paid inference, assign a bounded polarity/impact score from each headline, store the result, and expose the records

through a region-aware dashboard. The component separation prevents the dashboard from directly depending on the availability or latency of every feed and model provider.

1.2.1 Analytical Target

The analytical target is deliberately narrow. The model receives the title, not the full article. A resulting score estimates the polarity of the event or outcome stated in that title; it does not establish factual truth, article quality, causal impact, or public opinion. Sentiment analysis is context-dependent, and headline-only classification omits article context and may be ambiguous (Pang & Lee, 2008). The score is therefore a triage signal rather than a substitute for editorial judgment.

1.2.2 System Boundary

The system boundary begins at publisher-operated RSS/Atom endpoints and ends at the MongoDB-backed dashboard. Website crawling, article-body retrieval, claim verification, and user-level opinion measurement are outside the scope. The report treats configured values and source code as evidence of design behavior; it does not convert those configuration facts into measured service-level results.

2. Data Sources and Design Requirements

2.1 Publisher Sources

The producer configuration enumerates 19 endpoints, comprising 12 international and 7 Nepali publishers. The international set includes general news, public broadcasting, and international outlets; the Nepali set includes national publishers. Appendix A records the configured endpoint URLs. RSS is a publisher-supplied syndication format, and the feed inventory should be interpreted as configuration rather than a guarantee of endpoint availability (RSS Advisory Board, n.d.).

The producer interleaves the two regions in its polling list, fetches endpoints sequentially, and parses RSS/Atom responses with feedparser. Each request has a 30-second timeout. The loop sleeps for 60 seconds after a complete pass; therefore, the interval between the start of consecutive passes is the accumulated feed-request time plus the configured sleep, not a guaranteed 60-second polling interval.

2.2 Design Requirements

1. **Provenance.** Each record retains publisher, region, source link, publication time where supplied, and UTC ingestion time so that dashboard results remain attributable and temporally interpretable.
2. **Input robustness.** Parsing must tolerate heterogeneous RSS/Atom structures, malformed XML, absent timestamps, and optional excerpts. Recovered entries are usable only when the producer can form a record with a title and link.

3. **Pre-inference deduplication.** Repeated exact titles should be removed against both the current Spark batch and persisted MongoDB titles before model requests, avoiding repeated inference for the same title in the active consumer path.
4. **Regional language handling.** Region determines prompt and credential selection. Nepali instructions are written in English and allow Devanagari Nepali, Romanized Nepali, English, and mixed-language titles.
5. **Bounded model requests.** Requests contain no more than five titles from one region, use stable item identifiers and a constrained JSON response, and are paced to limit request bursts.
6. **Persistent and queryable output.** The stored document must retain the original source metadata alongside sentiment fields so the dashboard can filter by region and time without re-fetching the feeds.
7. **Data minimization and credential isolation.** Only the title is sent to an inference provider. Provider credentials are supplied through ignored local environment files rather than source code or report artifacts.

3. Data Contract and Sentiment Representation

Each producer event is a JSON record with the fields summarized in Table 1. The RSS excerpt is cleaned and capped at 6,000 characters when present; it remains available for display and coverage analysis but is excluded from the model request. Publisher time is taken from the published timestamp, falling back to the updated timestamp; ingestion time is assigned in UTC before publication to Kafka.

Table 1. News record fields and interpretation

Field group	Definition and use
title	Headline text; the sole article content sent to the sentiment provider.
text	Optional cleaned RSS excerpt, capped at 6,000 characters; retained but not scored.
link, source, region	Publisher URL, publisher label, and International or Nepali classification.
published_at	Publisher publication time, or RSS update time when publication time is absent; otherwise null.
ingested_at	UTC time assigned by the producer before Kafka publication.
sentiment / score	Positive / Neutral / Negative label in <i>sentiment</i> and numeric score in <i>sentiment_score</i> , in $[-100, +100]$; the API returns an integer and the stored field is numeric.
model / language	Provider model identifier (<i>sentiment_model</i>) and script-based language label (<i>sentiment_language</i>).
confidence	<i>sentiment_confidence</i> is stored as null; the response contract does not provide calibrated confidence.

The label mapping is deterministic: scores greater than or equal to +20 are Positive, scores less than or equal to -20 are Negative, and scores from -19 through +19 are Neutral. Records without a usable title are marked Unscored with a null score. The score range is an application convention, not a calibrated probability scale.

Language metadata is a simple script heuristic: a title containing Devanagari is labeled Nepali; other scripts are labeled English. Romanized Nepali may therefore receive an English metadata label even though the Nepali-region prompt accepts Romanized Nepali. The confidence field remains null; neither this schema nor the provider response implies a calibrated confidence estimate.

4. Storage Strategy and Component Selection

4.1 Event Buffer and Stream Processor

4.1.1 Apache Kafka

Apache Kafka provides the producer-consumer boundary. The *news_topic* topic is configured with six partitions and replication factor one. Kafka decouples sequential RSS polling from variable Spark and external-inference latency and supplies partitioned event delivery (Apache Software Foundation, n.d.-a). The Compose configuration does not mount a persistent Kafka data volume; Kafka is consequently treated as a short-term transport layer, not as the long-term archive.

4.1.2 Apache Spark Structured Streaming

Apache Spark Structured Streaming consumes Kafka records as incremental micro-batches and provides checkpointed query progress (Apache Software Foundation, n.d.-b). The application sets *maxOffsetsPerTrigger* to five and uses *foreachBatch* to perform MongoDB checks, inference orchestration, and writes. The active checkpoint volume is *spark-checkpoint-v2*, mounted at */tmp/spark-checkpoint*. A new checkpoint uses the configured earliest Kafka offsets; an existing checkpoint resumes its recorded progress.

4.2 Persistent Record Store

MongoDB stores one document per accepted headline and its analysis fields in *newsdb.sentiments*. A document model accommodates source-specific metadata without requiring a fixed relational join at ingestion; MongoDB indexes support title, region, ingestion-time, and region/publication-time queries (MongoDB, n.d.). Compose mounts *mongo-data* at */data/db*, preserving MongoDB data across ordinary container restarts. The title index is non-unique: duplicate suppression is application logic and not a database uniqueness constraint.

Environment files hold provider credentials and model settings. The repository supplies placeholder examples, while local *.env*, *.env.cloudflare*, and *.env.nepali* files are excluded from version control. No credential value is reproduced in this report. The Compose

configuration is intended for local development; it exposes services on host ports and does not configure Kafka or MongoDB authentication.

4.3 Component Rationale

Python and **feedparser** provide feed polling, parsing, timestamp extraction, record normalization, and Kafka publication. **Kafka with ZooKeeper** separates ingestion from downstream processing using a brokered event stream. **Spark Structured Streaming** implements checkpointed micro-batch consumption and coordinates duplicate filtering, region grouping, model calls, and persistence.

Cloudflare Workers AI is the primary inference provider; the configured default model identifier is *@cf/meta/llama-3.1-8b-instruct*. Its request schema constrains the response to JSON (Cloudflare, n.d.). **Gemini**, configured as *gemini-3.1-flash-lite*, is a fallback when the primary provider call or response validation fails; its structured-output schema specifies JSON fields (Google, n.d.). Provider fallback is sequential, not parallel key rotation.

MongoDB supports both persistent record storage and title lookup; **Streamlit**, with Pandas and Plotly, supplies regional story views and analytics; and **Docker Compose** defines the local service network, containers, and named data/checkpoint volumes. The dashboard reads MongoDB, not Kafka, and refreshes its fragment every ten seconds. These tools are complementary: Kafka is the event buffer, Spark the stream processor, MongoDB the queryable record store, and Streamlit the presentation layer.

Implementation paths relevant to the described behavior include *producer/news_producer.py*, *spark/sentiment_pipeline.py*, *spark/sentiment_stream.py*, *spark/sentiment_models.py*, and *dashboard/app.py*.

5. System Architecture

5.1 Architecture Overview

The architecture in Figure 1 distinguishes external publishers and inference services from the local Compose deployment. The Spark stage consults MongoDB and removes repeated current-batch titles before sending any model request. Inference responses return to Spark for validation and record association; scored records are then appended to MongoDB, from which the dashboard obtains its views.

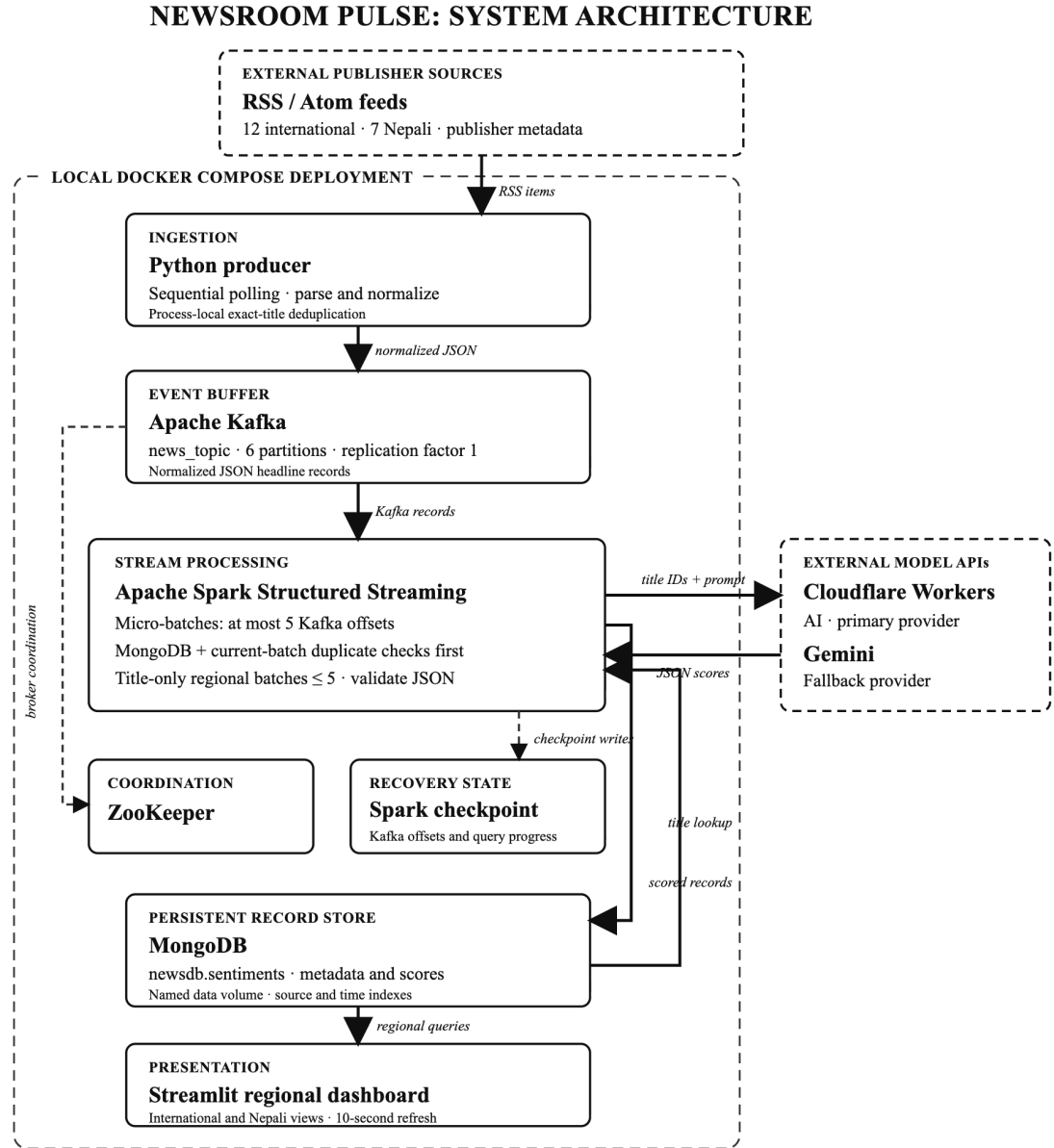


Figure 1. Newsroom Pulse architecture. Solid connectors denote data flow; dashed connectors denote broker coordination or checkpoint state. Duplicate title checks against MongoDB and the current micro-batch precede external inference.

6. Processing and Inference Method

1. **Poll and normalize.** The producer fetches each configured feed sequentially, recovers eligible entries from malformed responses, extracts title and link, assigns source and region, parses publication/update time, cleans the optional excerpt, and assigns UTC ingestion time.
2. **Suppress producer-local repeats.** An in-memory set prevents an exact title already sent by that producer process from being published again. This state resets on producer restart and is not the persistent deduplication boundary.
3. **Publish.** Accepted records are serialized as JSON and sent to Kafka topic *news_topic*. Six partitions are configured with replication factor one.
4. **Consume bounded offsets.** Spark reads Kafka incrementally with at most five offsets per trigger, parses the event schema, and records query progress in the checkpoint.
5. **Deduplicate before inference.** For each micro-batch, Spark collects the records to its driver, queries MongoDB for existing exact titles, and removes those matches and repeated titles within the batch. Only non-empty new titles proceed to scoring.
6. **Form regional requests.** Remaining records are grouped by region and divided into batches of at most five. The provider receives each title with a stable numeric identifier; the RSS excerpt and other article text are not sent.
7. **Validate and map results.** Cloudflare Workers AI is tried first. If the call or response validation fails, Gemini is attempted with the same region-specific credentials and prompt policy. The response must contain exactly one integer score per identifier, no duplicate or missing identifiers, and scores within $[-100, +100]$. Valid results are restored to the input order.
8. **Persist and present.** Spark attaches the label, score, model identifier, and language field and appends the enriched documents to MongoDB. Streamlit retrieves recent stories, source links, and region-specific summaries from MongoDB.

The international and Nepali prompts are separate. Both are written in English and require structured JSON. The Nepali prompt explicitly accepts Devanagari, Romanized Nepali, English, and mixed-language titles; it asks the model to interpret headline meaning rather than infer unstated article context. The application uses temperature zero for the primary request and a default six-second minimum interval between provider request starts. Pacing is process-local and does not guarantee compliance with an account-wide provider quota. Provider requests have a 30-second timeout.

7. Demonstration Protocol and Evaluation Plan

7.1 Demonstration Protocol

A demonstration can present the end-to-end path by preparing local environment files from their checked-in examples, starting the Compose services with *docker compose up --build -d*, and inspecting producer and Spark submission logs. The dashboard is available at <http://localhost:8501>. Its International and Nepali views expose source attribution, article links, publication time in Nepal time, headline score, and label. The Analytics view presents source counts, sentiment mix, and ingestion summaries. Container shutdown with *docker compose down* preserves MongoDB and Spark checkpoint volumes; removing volumes deletes that persisted state.

7.2 Evaluation Measures

The code configuration defines operational bounds but does not provide a ground-truth sentiment corpus or end-to-end latency telemetry. A defensible evaluation should report the following measures rather than infer them from request intervals or dashboard refresh frequency.

1. **Feed coverage:** successful HTTP fetches, parseable entries, recoveries, empty responses, and failures by endpoint and polling cycle.
2. **Freshness:** for each displayed story, compute latency from ingestion to dashboard visibility; report median and 95th-percentile latency, with publication-to-ingestion age reported separately.
3. **Duplicate/API efficiency:** count candidate titles, MongoDB matches, within-batch duplicates, and titles submitted to a provider; report duplicate suppression before inference.
4. **Sentiment quality:** obtain human annotations stratified by region, script, publisher, and label. Report macro-F1 for labels, agreement statistics, and score error against an explicitly defined annotated scale. Review Romanized Nepali separately.
5. **Provider behavior:** report primary-provider success, fallback frequency, invalid-response rate, request latency, rate-limit errors, and cost using provider telemetry.
6. **Capacity and resilience:** measure consumer backlog and throughput while varying input rates; test worker, broker, model-provider, and MongoDB failures separately.

7.3 Annotation Protocol and Measurement

Sentiment evaluation should use a human-annotated reference set sampled across region, publisher, script, and predicted label, rather than relying on an unstratified convenience sample. At least two annotators should independently label each selected title using a written rubric that defines the target as stated headline impact, distinguishes neutral or ambiguous reports from positive/negative events, and treats score magnitude as ordinal rather than

probabilistic. Adjudication should resolve disagreements before the reference labels are used. Annotators should not see model outputs during initial labeling to reduce confirmation bias.

For categorical results, report per-class precision and recall, macro-averaged F1, and inter-annotator agreement. For numeric scores, report an ordinal agreement statistic such as weighted Cohen’s κ together with mean absolute error against an explicitly defined human scale. Results should be disaggregated by region and script; aggregate scores can conceal systematic errors in Romanized Nepali or source-specific vocabulary. The reference set, prompt version, model identifier, and evaluation date should be retained so that later prompt or provider changes can be compared reproducibly.

Processing freshness should be separated from source freshness. For record i , define pipeline latency as $L_i = t_{\text{visible},i} - t_{\text{ingested},i}$; report median and 95th-percentile latency, while publication-to-ingestion age is reported as a separate feed-delay measure. For throughput experiments, record Kafka backlog, offsets consumed per trigger, titles removed by each duplicate stage, provider request latency, fallback outcome, and CPU/memory use while applying a controlled event rate. Tests should isolate broker, model-provider, Spark-worker, and MongoDB failures instead of changing several components simultaneously.

Several validity threats must be considered. The 19 configured feeds are not a probability sample of global or Nepali news, and publisher output frequency can dominate regional averages. Headline-only analysis may assign a polarity that a full article would qualify. Prompt language and the selected generative model can introduce language- and topic-dependent bias. These limits motivate stratified annotation and source-level reporting; they preclude interpreting an unqualified dashboard mean as a population-level measure of public opinion.

7.4 Architectural Limitations

The active Spark query uses one configured worker and collects each micro-batch to the driver before MongoDB lookups and model calls. This centralizes provider pacing but limits horizontal inference scaling and can increase driver memory pressure. Kafka replication factor one does not provide broker-failure tolerance, and no durable Kafka volume is configured. The title index is not unique; exact-string matching can miss spelling or punctuation variants and does not establish global exactly-once processing. Sequential feed polling makes source-level request timeouts additive to the cycle duration. Provider pacing is process-local, while MongoDB and Kafka have no authentication in the local Compose configuration.

Sentiment interpretation also has limits: a short headline may omit context, cross-lingual model behavior needs human evaluation, the language label is script-based, and the numeric score is not calibrated confidence. These constraints should accompany any interpretation of dashboard averages or polarity distributions.

8. Conclusion

Newsroom Pulse connects RSS collection, Kafka buffering, Spark deduplication and title-only scoring, MongoDB persistence, and Streamlit presentation for international and Nepali headlines. Single-worker driver inference, process-local pacing, exact-title matching, and a single-replica Kafka topic limit resilience.

Appendix A. Configured RSS Feed Inventory

The endpoint list reproduces the producer configuration. Publisher endpoint availability and content policies remain under publisher control.

Table 2. Configured RSS source inventory

Region	Publisher	Configured feed endpoint
International	BBC	https://feeds.bbc.co.uk/news/rss.xml
Nepali	Onlinekhabar	https://www.onlinekhabar.com/feed
International	CBS News	https://www.cbsnews.com/latest/rss/main
Nepali	Setopati	https://www.setopati.com/feed
International	The New York Times	https://rss.nytimes.com/services/xml/rss/nyt/HomePage.xml
Nepali	Nepal News	https://nepalnews.com/feed
International	Google News (U.S.)	https://news.google.com/rss?hl=en-US&gl=US&ceid=US:en
Nepali	Ratopati	https://www.ratopati.com/feed
International	NPR	https://feeds.npr.org/1003/rss.xml
Nepali	Nepal Press	https://www.nepalpress.com/feed/
International	The Guardian	https://www.theguardian.com/world/rss
Nepali	The Himalayan Times	https://www.thehimalayantimes.com/rssFeed/15
International	Al Jazeera	https://www.aljazeera.com/xml/rss/all.xml
Nepali	The Kathmandu Post	https://kathmandupost.com/rss/
International	Deutsche Welle	https://rss.dw.com/rdf/rss-en-all
International	France 24	https://www.france24.com/en/rss
International	CNBC	https://www.cnbc.com/id/100003114/device/rss/rss.html
International	NBC News	https://feeds.nbcnews.com/nbcnews/public/news
International	ABC News	https://abcnews.go.com/abcnews/topstories

References

- Apache Software Foundation. (n.d.-a). *Apache Kafka documentation*.
<https://kafka.apache.org/documentation/>
- Apache Software Foundation. (n.d.-b). *Structured Streaming programming guide (Apache Spark 3.5.6)*. <https://spark.apache.org/docs/3.5.6/structured-streaming-programming-guide.html>
- Cloudflare. (n.d.). *JSON mode*. Workers AI documentation.
<https://developers.cloudflare.com/workers-ai/features/json-mode/>
- Google. (n.d.). *Structured output*. Gemini API documentation. <https://ai.google.dev/gemini-api/docs/generate-content/structured-output>
- MongoDB. (n.d.). *Data modeling*. MongoDB Manual. <https://www.mongodb.com/docs/manual/data-modeling/>
- Pang, B., & Lee, L. (2008). Opinion mining and sentiment analysis. *Foundations and Trends in Information Retrieval*, 2(1–2), 1–135. <https://doi.org/10.1561/15000000011>
- RSS Advisory Board. (n.d.). *RSS 2.0 specification*. <https://www.rssboard.org/rss-specification>